

```
[anil@shrishti Articles]$ fdisk -l
bash: fdisk: command not found
[anil@shrishti Articles]$ sudo fdisk -l

Disk /dev/sda: 500.1 GB, 500107862016 bytes
255 heads, 63 sectors/track, 60801 cylinders
units = cylinders of 16065 * 512 = 8225280 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x6e06b8da

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1 *          1          997      6080272    82  Linux swap / Solaris
/dev/sda2            998       6469   43953848    83  Linux
/dev/sda3           6470      11941   43953848    83  Linux
/dev/sda4          11942      60801  392467956    83  Linux
[anil@shrishti Articles]$
```

Figure 1: Partition listing by fdisk

```
[anil@shrishti Articles]$ sudo ./part_info /dev/sda

DOS type Partition Table of /dev/sda:
  # Start (H/C/S)   End (H/C/S) Type  StartSec TotSec
1:1 [ 1/ 1/ 1] (254/ 997/63) 62    63 16015742
2:0 [ 0/ 998/ 1] (254/1024/63) 83 16015805 87907680
3:0 (254/1024/63) (254/1024/63) 83 103924485 87907680
4:0 (254/1024/63) (254/1024/63) 83 191832165 784935908

Re-computed Partition Table of /dev/sda:
  # Start (H/C/S)   End (H/C/S) Type  StartSec TotSec
1:1 [ 1/ 1/ 1] (254/ 997/63) 62    63 16015742
2:0 [ 0/ 998/ 1] (254/ 6469/63) 83 16015805 87907680
3:0 [ 0/ 6470/ 1] (254/11941/63) 83 103924485 87907680
4:0 [ 0/11942/ 1] (254/60801/63) 83 191832165 784935908
[anil@shrishti Articles]$
```

Figure 2: Output of ./part_info

partitioning, and be able to maintain the same partition table formats, especially for the most prevalent DOS-type partition tables, which heavily depend on this simplistic geometry. Note the computation of cylinder size (255 heads * 63 sectors / track * 512 bytes / sector = 8225280 bytes) in the third line and then the demarcation of partitions in units of complete cylinders.

DOS-type partition tables

This brings us to the next important topic: understanding DOS-type partition tables. But first, what is a partition, and why should we partition? A hard disk can be divided into one or more logical disks, each of which is called a partition. This is useful for organising different types of data separately, for example, different operating system data, user data, temporary data, etc. So, partitions are basically logical divisions and need to be maintained by meta data, which is the partition table. A DOS-type

partition table contains four partition entries, each a 16-byte entry. Each of these entries can be depicted by the following 'C' structure:

```
typedef struct
{
    unsigned char boot_type; // 0x00 -
    Inactive; 0x00 - Active (Bootable)
    unsigned char start_head;
    unsigned char start_sec;
    unsigned char start_cyl_hi;
    unsigned char start_cyl;
    unsigned char part_type;
    unsigned char end_head;
    unsigned char end_sec;
    unsigned char end_cyl_hi;
    unsigned char end_cyl;
    unsigned long abs_start_sec;
    unsigned long sec_in_part;
} PartEntry;
```

This partition table, followed by the two-byte signature 0xAA55, resides at the end of the disk's first sector, commonly known as the Master Boot

Record (MBR). Hence, the starting offset of this partition table within the MBR is $512 - (4 * 16 + 2) = 446$. Also, a 4-byte disk signature is placed at offset 440. The remaining top 440 bytes of the MBR are typically used to place the first piece of boot code, that is loaded by the BIOS to boot the system from the disk. The `part_info.c` listing contains these various definitions, along with code for parsing and printing a formatted output of the partition table.

From the partition table entry structure, it can be noted that the start and end cylinder fields are only 10 bits long, thus allowing a maximum of 1023 cylinders only. However, for today's huge hard disks, this is in no way sufficient. Hence, in overflow cases, the corresponding <head, cylinder, sector> triplet in the partition table entry is set to the maximum value, and the actual value is computed using the last two fields: the absolute start sector number (`abs_start_sec`) and the number of sectors in this partition (`sec_in_part`). The code for this too is in `part_info.c` (download the file from http://www.linuxforu.com/article_source_code/dec11/partinfo.tar.gz):

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

#define SECTOR_SIZE 512
#define MBR_SIZE SECTOR_SIZE
#define MBR_DISK_SIGNATURE_OFFSET 440
#define MBR_DISK_SIGNATURE_SIZE 4
#define PARTITION_TABLE_OFFSET 446
#define PARTITION_ENTRY_SIZE 16 //
sizeof(PartEntry)
#define PARTITION_TABLE_SIZE 64 //
sizeof(PartTable)
#define MBR_SIGNATURE_OFFSET 510
#define MBR_SIGNATURE_SIZE 2
#define MBR_SIGNATURE 0xAA55
#define BR_SIZE SECTOR_SIZE
#define BR_SIGNATURE_OFFSET 510
#define BR_SIGNATURE_SIZE 2
#define BR_SIGNATURE 0xAA55
```